# MANUAL DE USUARIO

Taller Integrador de Repaso e Integración
Laboratorio de Programación (LPR) — 5.º año

**Programa:** Suma recursiva · Búsqueda secuencial · Intercambio con punteros

**Estudiante:** Federico Mattia
**Institución:** E.E.S.T. N.º 20 “Bangtan Sonyeondan” — Vicente López
**Archivo principal:** src/main.cpp

Guía práctica para compilar, ejecutar y comprender los resultados del programa.

# Contenido del manual


| Sección | Qué vas a encontrar |
|---|---|
| 1. Presentación | Objetivo y alcance del programa. |
| 2. Requisitos | Qué necesitás para compilar y ejecutar. |
| 3. Puesta en marcha | Pasos para abrir, compilar y correr el código. |
| 4. Uso del programa | Qué muestra cada reto y cómo interpretar la salida. |
| 5. Conceptos clave | Recursividad, arreglos, direcciones de memoria y punteros. |
| 6. Problemas frecuentes | Soluciones a errores comunes. |


# 1. Presentación

Este manual explica cómo utilizar el programa de consola desarrollado en C++ para la Actividad 6 de Laboratorio de Programación. Al ejecutarlo, el programa realiza tres demostraciones: calcula una suma mediante recursividad, busca un número dentro de un arreglo y cambia dos valores utilizando punteros.

El programa está preparado para ejecutar los tres retos en orden, sin pedir datos al usuario. Los valores que utiliza están definidos directamente en el archivo **src/main.cpp**.

## Objetivos

- Observar cómo una función puede llamarse a sí misma para resolver un problema.
- Encontrar un dato dentro de un arreglo mediante una búsqueda secuencial.
- Comprender cómo los punteros permiten modificar variables a través de sus direcciones de memoria.

# 2. Requisitos

- Una computadora con Windows, Linux o macOS.
- Un compilador de C++ instalado, por ejemplo g++ (MinGW-w64 en Windows) o el compilador integrado en un IDE.
- El archivo fuente **main.cpp** guardado dentro de la carpeta del proyecto.

No se requieren conexión a Internet, base de datos ni archivos adicionales para ejecutar este programa.

# 3. Puesta en marcha

## Opción A: desde Visual Studio Code

- Abrí Visual Studio Code y seleccioná **Archivo** → **Abrir carpeta** para abrir el proyecto.
- Verificá que el archivo **src/main.cpp** contenga el código del programa.
- Abrí una terminal desde **Terminal** → **Nueva terminal**.
- Compilá el programa y después ejecutá el archivo generado. Los comandos dependen del sistema operativo y del compilador.

## Ejemplo con g++ en una terminal de Windows (MinGW):

```
g++ src/main.cpp -o taller.exe
```

```
.\taller.exe
```

## Ejemplo con g++ en Linux o macOS:

```
g++ src/main.cpp -o taller
./taller
```

Si el archivo **main.cpp** está en la carpeta actual y no dentro de **src**, reemplazá **src/main.cpp** por **main.cpp** en el comando.

Importante: si el compilador indica que no reconoce el comando **g++**, revisá que el compilador esté instalado y que su carpeta esté agregada a la variable PATH.

# 4. Uso del programa

Una vez iniciado, el programa muestra un encabezado y ejecuta automáticamente los tres retos. No hace falta ingresar información ni presionar opciones del menú.

## Reto 1 — Suma recursiva

El programa llama a **sumaRecursiva(5)**. La función suma el valor actual de **n** con el resultado de la llamada siguiente, que utiliza **n - 1**. Cuando **n** llega a cero o a un número negativo, devuelve 0 y finaliza la cadena de llamadas.

```
Suma acumulada de 1 hasta 5: 15
```

El cálculo equivale a **5 + 4 + 3 + 2 + 1 = 15**. El caso base **if (n <= 0) return 0;** evita que la función continúe llamándose indefinidamente.

## Reto 2 — Búsqueda secuencial en un arreglo

El programa crea un arreglo de 10 números y busca el valor **18**, comparándolo con cada elemento desde el índice 0. Cuando lo encuentra, informa el índice y la dirección de memoria de ese elemento. Luego utiliza **break** para terminar el ciclo.

```
Arreglo: {12, 45, 78, 90, 122, 5, 18, 33, 67, 99}
Valor buscado: 18
Resultado: hallado en el índice [6]
```

Los índices de un arreglo en C++ empiezan en 0; por eso, el séptimo elemento está en el índice 6. La dirección que aparece junto a “Dirección RAM Hexadecimal” puede cambiar entre ejecuciones. El formato exacto depende del entorno y del compilador; normalmente se muestra como una dirección de memoria.

Si se cambia **numeroBuscar** por un número que no aparece en el arreglo, el programa informa que el valor no existe.

## Reto 3 — Intercambio de valores con punteros

Se crean dos variables: **x = 100** e **y = 500**. La función **intercambiarValores(&x, &y)** recibe las direcciones de esas variables y modifica los valores originales.

| Momento | X | Y |
|---|---|---|
| Antes de la función | 100 | 500 |
| Después de la función | 500 | 100 |

Dentro de la función, la variable temporal guarda el valor al que apunta **ptrA**; luego se copia el valor apuntado por **ptrB** a la primera variable y, por último, se asigna el valor temporal a la segunda. El asterisco en ***ptrA** y ***ptrB** permite acceder al valor almacenado en esas direcciones.

# 5. Cómo interpretar la salida

La salida puede variar en el texto de las direcciones de memoria, pero los resultados de los tres retos deberían ser los siguientes:

```
=== RETO 1: SUMA RECURSIVA ===
Suma acumulada de 1 hasta 5: 15

=== RETO 2: BÚSQUEDA EN MEMORIA CONTIGUA ===
[SISTEMA] Valor 18 hallado en el indice [6]
```

```
          Direccion RAM Hexadecimal: (dirección variable)

=== RETO 3: INTERCAMBIO DE CELDAS ===
Valores previos  -> X: 100 | Y: 500
Valores actuales -> X: 500 | Y: 100
```

La dirección de memoria se representa de forma orientativa en el ejemplo anterior: el programa mostrará una dirección real del equipo, no el texto “(dirección variable)”.

# 6. Conceptos clave

| Concepto | Explicación sencilla |
|---|---|
| Recursividad | Una función se llama a sí misma con un problema más pequeño hasta alcanzar un caso base. |
| Caso base | Condición que detiene las llamadas recursivas. En este programa es n <= 0. |
| Arreglo (array) | Conjunto de elementos del mismo tipo almacenados en posiciones consecutivas. |
| Índice | Número que identifica la posición de un elemento. En C++ comienza en 0. |
| Dirección de memoria | Ubicación en memoria donde se encuentra una variable o elemento. |
| Puntero | Variable que almacena una dirección de memoria. |
| & | En este código, permite obtener la dirección de una variable o elemento. |
| * | En una expresión como *ptrA, accede al valor guardado en la dirección a la que apunta el puntero. |

# 7. Problemas frecuentes

| Problema | Qué revisar |
|---|---|
| “g++ no se reconoce…” | Instalá un compilador de C++ y verificá que su carpeta esté configurada en PATH. Cerrá y volvé a abrir la terminal después de cambiarlo. |
| No se encuentra src/main.cpp | Comprobá que la terminal esté ubicada en la carpeta raíz del proyecto. Si el archivo está en otra ubicación, corregí la ruta del comando. |
| El programa se compila, pero no se ejecuta | Verificá que el nombre del ejecutable coincida con el del comando de ejecución y que la compilación haya terminado sin errores. |
| La dirección de memoria es distinta | Es normal: las direcciones pueden variar entre ejecuciones y equipos. |
| El número buscado no aparece | Revisá el valor de numeroBuscar y los elementos del arreglo vectorDatos. |

# 8. Recomendaciones de uso

- No modifiques el código original sin guardar una copia si necesitás conservar la versión de la actividad.
- Para experimentar con la búsqueda, cambiá **numeroBuscar** por un valor que esté o no esté en el arreglo.
- Para probar el intercambio, cambiá los valores iniciales de **x** e **y** y comprobá que se inviertan.
- Para probar la recursividad, modificá **num** con valores pequeños y positivos. La función también devuelve 0 para entradas menores o iguales a cero.

# 9. Resultado esperado

Si el programa se compila correctamente, debe terminar mostrando la suma 15, encontrar el número 18 en el índice 6 y dejar las variables intercambiadas (X = 500, Y = 100). Finalmente, **return 0** indica que la ejecución terminó normalmente.

Este programa es una demostración educativa de conceptos básicos de C++. No guarda datos en archivos ni cambia configuraciones del sistema.

Documento elaborado a partir del código fuente proporcionado para la Actividad 6 de Laboratorio de Programación.